

Corrections techniques — Cahier de Charge

Orchestrateur Mobile Money Open Source — v1.2

Document complémentaire issu de la revue technique de la v1.2. Objectif : verrouiller les points critiques avant le démarrage de l'implémentation.

1. Idempotence côté provider — CRITIQUE

La spécification garantit l'idempotence au niveau de l'orchestrateur, mais elle doit également garantir qu'un retry ou un fallback ne provoque pas un second débit chez un provider.

Correction recommandée : ajouter au contrat provider une capacité d'idempotence et utiliser une clé provider persistée par tentative.

```
supportsIdempotency(): boolean
initiate(...)
verify(...)
```

La clé d'idempotence provider doit être déterministe ou persistée dans PaymentAttempt. Elle ne doit jamais changer lors d'un retry involontaire.

2. Unknown / timeout et risque de double débit — CRITIQUE

Après un timeout, l'orchestrateur ne sait pas si le provider a reçu et exécuté la demande. Le cahier impose déjà verify() avant fallback ; cette règle doit être renforcée pour empêcher un second débit lorsque le résultat du premier provider reste incertain.

Règle : aucun fallback automatique tant que l'état du premier provider n'est pas suffisamment confirmé. Si verify() reste unknown/pending, le paiement reste dans cet état et poursuit son polling.

Le provider doit clairement distinguer : succeeded, confirmed_failed, pending et unknown.

3. Fallback après verify() = failed — CRITIQUE

La v1.2 autorise un fallback après verify() = failed. Cette décision doit dépendre de la qualité de la confirmation fournie par le provider. Un simple 'failed' ambigu ne doit pas déclencher un second débit chez un autre provider.

Correction : le contrat provider doit permettre de représenter un échec confirmé (confirmed_failed) distinct d'un résultat incertain (unknown). Seul un échec réellement confirmé peut autoriser le fallback selon la politique d'erreur.

4. supports() : éviter un appel réseau systématique

Le contrat actuel définit supports(params) comme Promise. Pour le MVP, il est préférable que supports() s'appuie sur les capacités locales configurées et ne déclenche pas un appel réseau à chaque paiement.

```
supports(params): boolean
```

Les capacités couvrent notamment pays, réseaux, devises, montants min/max et opérations. L'appel réseau réel intervient lors de initiate() / verify().

5. Montants financiers : unités mineures

Pour le moteur interne, il est recommandé de représenter les montants en unités mineures avec un entier (par exemple BIGINT), plutôt que de faire reposer les calculs métier sur decimal.

```
amount_minor BIGINT
currency CHAR(3)
```

L'API peut continuer à exposer un format adapté à la devise. Le moteur conserve une représentation entière pour les comparaisons et calculs financiers.

6. Clarifier expired et unknown

La v1.2 utilise 'expired/unknown' à expiration. Ces deux états doivent être explicitement distincts.

État	Signification
unknown	Le système ne sait pas avec certitude ce qui s'est produit chez le provider.
expired	La fenêtre métier de confirmation est dépassée sans confirmation finale.

Il faut également définir le comportement lorsqu'un webhook succeeded arrive après expiration.

7. Webhook tardif après expired/unknown

Scénario à spécifier : Payment = expired, puis le provider envoie un webhook succeeded. Le système ne doit pas modifier aveuglément l'état final sans politique explicite.

Correction recommandée : enregistrer l'événement tardif, l'associer au Payment, produire un AuditLog et orienter le cas vers une logique de reconciliation / traitement manuel future. La v1 doit au minimum définir explicitement cette règle.

8. Secret des webhooks marchands

Le champ secret? de POST /api/v1/webhooks est à renforcer. Le système devrait générer le secret plutôt que dépendre d'un secret fourni par le marchand.

- POST /webhooks
- secret généré
- affiché une seule fois
- stocké de manière sécurisée
- utilisé pour HMAC-SHA256

9. Sécurité du Dashboard

La spécification indique un login admin local. Ajouter explicitement les protections suivantes : session sécurisée, cookies HttpOnly/Secure, protection CSRF, rate limiting du login, backoff/lockout, reset de mot de passe et Argon2 pour le hash du mot de passe. Le 2FA peut rester hors MVP ou être prévu pour une évolution ultérieure.

10. API Keys

Ajouter une règle de cycle de vie : une clé API complète est affichée uniquement lors de sa création. Après cela, seule son empreinte/hash et son prefix sont conservés/affichés. La révocation doit être immédiate.

Formats recommandés : mg_live_ et mg_test_.

11. Ordre de développement recommandé

1. Domain model
2. State Machine + invariants
3. Provider Contract
4. MockProvider
5. PostgreSQL + contraintes

6. Payment Engine
7. Idempotency + concurrence
8. Webhooks
9. Queue / polling / expiration
10. Premier provider réel
11. API REST
12. Dashboard
13. Providers supplémentaires

12. Checklist avant démarrage

- Idempotence provider définie et testée.
- Retry provider-safe défini.
- Fallback après timeout/unknown interdit tant que le résultat n'est pas suffisamment confirmé.
- confirmed_failed distingué de unknown.
- supports() local et basé sur capabilities.
- Montants internes définis en unités mineures.
- expired et unknown clairement séparés.
- Webhooks tardifs spécifiés.
- Secrets webhook générés et protégés.
- Dashboard admin protégé contre brute-force/CSRF.
- API keys non récupérables après création.
- Tests de concurrence et double débit automatisés.

Conclusion : la v1.2 constitue une bonne base technique. Ces corrections visent surtout à verrouiller les invariants financiers et les cas ambigus avant l'implémentation du Payment Engine.